Latency after the request/reply rewrite

latency_claude_3 · iteration 3 · ten in-place yaw steps, latency_bag2

This report answers *latency_real_report_2*, which is the previous real report and the brief for everything below. It asked for the informational latency (first motor movement minus the step change in `cmd_vel`), the full latency (reaching 95% of the setpoint minus the same reference), and graphs along the lines of the previous latency report. The reports it follows are *latency_claude_1* (the course runs) and *latency_claude_2* (the board-B echo test), and the numbers from both are carried forward here for comparison.

The headline is that the rewrite worked, and by more than the structural argument predicted. What is left is no longer a communications problem.

The two numbers that were asked for

Informational latency — 56 ± 23 ms (n = 10) at the finest detection threshold the encoder supports. Two estimators that need no threshold agree: the release edges give 38 ± 9 ms, and a first-order-plus-dead-time fit against the rate-limited command gives 41 ± 18 ms (45 ± 12 ms for the steps of 0.5 rad/s and above, $r^2 = 0.99$). Call it **40–55 ms**, of which about 15 ms is host-side and unavoidable at 100 Hz.

Full latency to 95% of the command — 704 ± 94 ms (n = 10, range 595–919 ms). Only about 40 ms of that is transport. The acceleration limiter accounts for 33–233 ms depending on amplitude, and everything remaining — roughly 560 ms on average — is the motor loop's own rise time.

What changed, and what was measured

Per *latency_real_report_2*: board B now pushes real wheel angular velocities to board A from inside its PID loop every `DATA_SEND_RATE_MS` (5 ms); board A never prints on its own; the hardware interface's `read()` writes a single `"g\n"` and then reads the waiting line. `update_rate` is 100 Hz, `velocity_rolling_window_size` is 3, and `angular.z.max_acceleration` is 3.0 rad/s².

`latency_bag2` is 51.6 s holding ten in-place rotations — ±0.1, ±0.2, ±0.5, ±0.7 and ±0.7 rad/s — each held for about 2.5 s and then released to zero. That gives twenty usable edges: ten rises and ten releases. No linear motion was commanded at any point.

Channel	Rate in the bag	Note
`diffbot_base_controller/cmd_vel`	31.3 Hz	published by rqt; the header stamp is unset, so the bag receive time is the only usable reference and all latencies below are measured from it
`dynamic_joint_states`	100.0 Hz	raw wheel velocities, exactly what `read()` wrote into the state interface
`diffbot_base_controller/odom`	100.0 Hz	the same data after the rolling mean
`imu/data`	19.6 Hz	the UM7, at the driver's default rate — see *Which signal is freshest*
`odometry/filtered/local`	30.0 Hz	the EKF, which is what nav2 consumes

The 100 Hz loop held: the interval between consecutive `dynamic_joint_states` messages has a median of 10.00 ms, a 99th percentile of 10.87 ms and a maximum of 11.47 ms across 5151 cycles, with not one cycle over 15 ms. `/rosout` carries no overrun warning, no `Backlog` line and no read failure for the whole run.

Writing `"g"` and reading a reply fits inside a 10 ms budget with room to spare, which was the open question going in.

Informational latency

The brief defines this as the time of first motor movement minus the time of the step change in `cmd_vel`. Measured that way it contains *both* directions of the link — host to A to B on the way out, B to A to host on the way back — plus whatever the motor itself takes to break away. It is an upper bound on the actuation dead time, and it is also precisely what the controller experiences, which is the number that matters.

The answer depends on where you put the threshold, so here is the whole curve

The wheel velocity arrives quantised: the reported value moves in steps of about 0.0565 rad/s per wheel, which is 0.0109 rad/s once converted to a yaw rate. Below roughly one quantum there is nothing left to detect; above it, you stop measuring when motion started and start measuring how hard the robot accelerates. Both ends of that trade are shown rather than one number being picked quietly.

Detection threshold	Mean onset, rise edges	sd	What it is really measuring
0.5 quanta (0.0054 rad/s)	56.0 ms	23.3	below the encoder resolution; identical to the 1-quantum result because no sample falls between them
1.0 quanta (0.0109 rad/s)	**56.0 ms**	**23.3**	**the finest honest threshold: one encoder count of motion**
1.5 quanta (0.0163 rad/s)	75.9 ms	17.6	starts to fold in acceleration
2.5 quanta (0.0272 rad/s)	91.9 ms	18.4	mostly acceleration
5.0 quanta (0.0545 rad/s)	152.9 ms	90.8	entirely acceleration; the 0.1 rad/s steps barely reach this at all, hence the sd of 91 ms

Two estimators that do not need a threshold

The release edges. When the command drops back to zero the robot is already turning, so the encoder is already producing counts every 5 ms and there is no breakaway to wait for: the first deceleration shows up in the very next reported sample. Those edges give **38 ± 9 ms** (threshold 1.5 quanta, which is the right one here because the signal is already fluctuating by about a quantum while moving). The gap between 56 ms on the rises and 38 ms on the releases is the cost of starting from rest — stiction plus the time to accumulate a first encoder count — and it is physical, not informational.

The model fit. Fitting *first order plus dead time* to each step, with the plant input taken as the command *after* the acceleration limiter rather than the raw step, separates transport from the ramp cleanly. Dead time comes out at **41 ± 18 ms** over all ten and **45 ± 12 ms** over the six steps of 0.5 rad/s and above, where the fit is tight (mean $r^2 = 0.991$). The small steps fit worse ($r^2 = 0.92$–0.95) because a first-order model cannot represent their overshoot.

Where the 40–55 ms sits

Roughly 15 ms is host-side and structural at 100 Hz: up to 10 ms for the controller to pick up a new `cmd_vel`, and one cycle of lag because `read()` writes `"g"` and then reads immediately, so it collects the reply to the *previous* cycle's request.

Up to 5 ms is board B's push period, plus a few ms for the CP2102 round trip on `/dev/ttyUSB0` and board A's `loop()` cadence.

The remainder, on the order of 20 ms, is the ESP-NOW hop and the motor's own electrical and mechanical dead time. That part is not addressable in software.

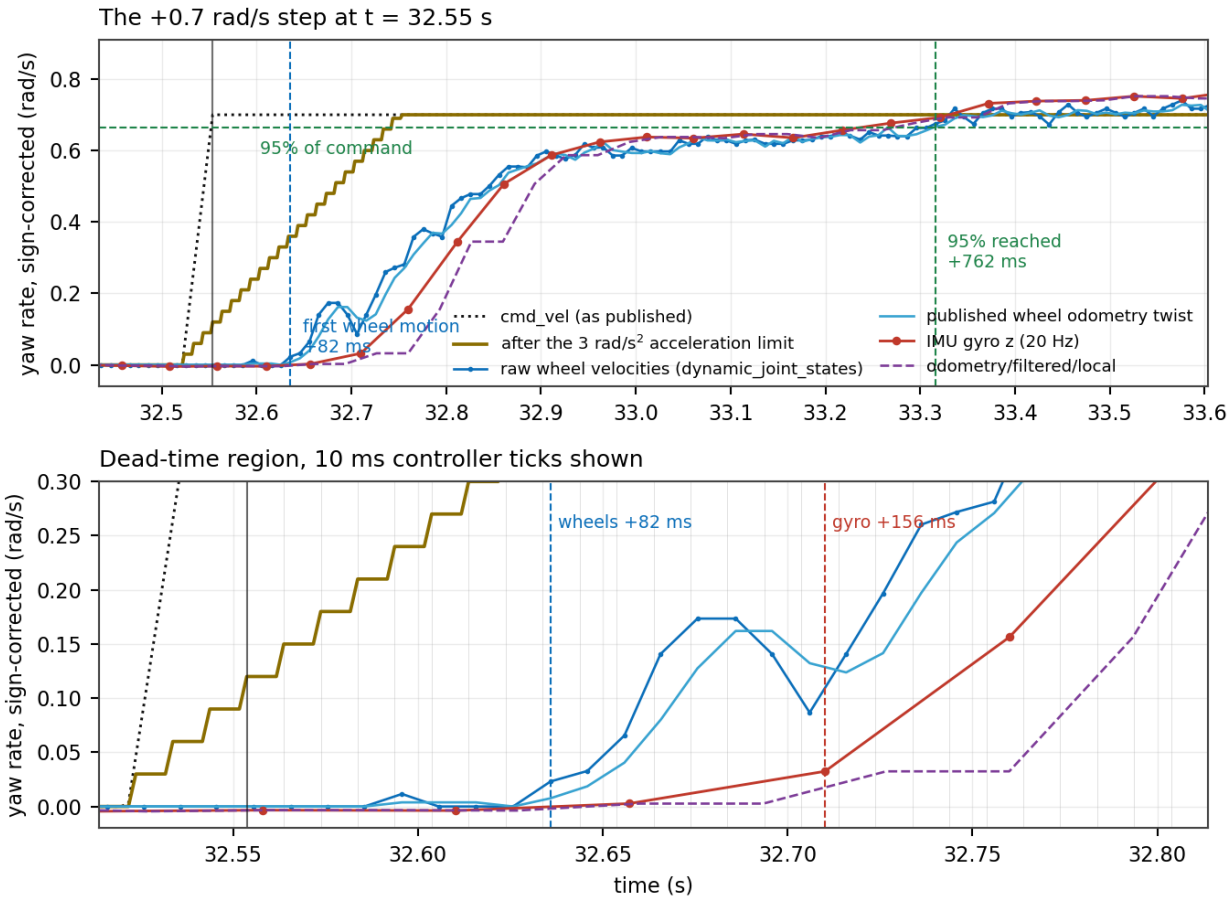


Figure 1. One step, the +0.7 rad/s at t = 32.55 s, with every velocity signal in the system overlaid and the sign flipped so that all of them rise. The gold trace is the command after diff_drive_controller's 3.0 rad/s² acceleration limit, which is what the hardware actually receives — note that it takes 233 ms to deliver the full 0.7 rad/s. The lower panel is the same data zoomed into the dead-time region with the 10 ms controller ticks drawn in. The raw wheels (dark blue) and the published odometry (light blue) are nearly on top of each other; the gyro (red) and the EKF (purple dashed) are visibly behind, which is the subject of *Which signal is freshest* below.

Full latency, and what it is made of

Reaching 95% of the commanded value takes **704 ± 94 ms**. Transport is about 40 ms of that. The rest splits into two pieces, one of which is pure configuration.

Step	Informational (1 quantum)	Full, to 95%	Limiter ramp |A| / 3.0	Full minus ramp	Overshoot	Fitted τ
+0.1	37 ms	667 ms	33 ms	634 ms	22.0%	260 ms
–0.1	41 ms	660 ms	33 ms	627 ms	10.8%	300 ms
+0.2	59 ms	919 ms	67 ms	852 ms	11.7%	390 ms
–0.2	90 ms	751 ms	67 ms	684 ms	9.8%	370 ms
+0.5	39 ms	659 ms	167 ms	492 ms	9.8%	190 ms
–0.5	92 ms	752 ms	167 ms	585 ms	8.1%	150 ms
+0.7	82 ms	762 ms	233 ms	529 ms	6.1%	150 ms
–0.7	35 ms	635 ms	233 ms	401 ms	5.1%	130 ms
+0.7	45 ms	595 ms	233 ms	361 ms	5.3%	130 ms
–0.7	41 ms	640 ms	233 ms	407 ms	3.2%	150 ms
mean	**56 ms**	**704 ms**	—	**557 ms**	**9.2%**	**222 ms**

Two things in that table are worth stating plainly. The steady-state gain is **1.004 ± 0.006** for the steps of 0.5 rad/s and above, so the loop has no steady-state error and the integral term is doing its job. And the 95% figures for the 0.1 and 0.2 rad/s steps should be treated as soft: 95% of 0.1 rad/s is a band narrower than the encoder quantum in yaw rate, so those two rows are reporting quantisation as much as dynamics. The 10–90% rise time is the sturdier measure at low amplitude, and it is 375 ± 81 ms for $|A| \geq 0.5$ against 460 ± 148 ms overall.

The acceleration limiter is quietly acting as an input shaper

Overshoot falls monotonically with amplitude: 22% at 0.1 rad/s down to 3–6% at 0.7. That is the opposite of the usual intuition, and it has a clean explanation. The limiter ramp lasts |A| / 3.0 seconds, so it is *proportional* to amplitude, while the closed-loop time constant is not. At 0.7 rad/s the ramp is 233 ms against a τ of 130 ms, so the command is spread over almost two time constants and the loop never sees a sharp edge. At 0.1 rad/s the ramp is 33 ms against a τ of 260 ms, so the loop sees very nearly an ideal step and rings.

This is the quantitative form of the observation in *latency_real_report_2* that raising P helps the large steps but makes small ones oscillate. The two effects cannot be fully separated from this dataset, because τ also grows at low amplitude (260–390 ms below 0.2 rad/s against 130–150 ms above 0.5), most likely from stiction. A test that varies `angular.z.max_acceleration` at a *fixed* step amplitude would separate them in one run, and that is the cheapest next experiment available.

All ten rise edges, each normalised by its own commanded value

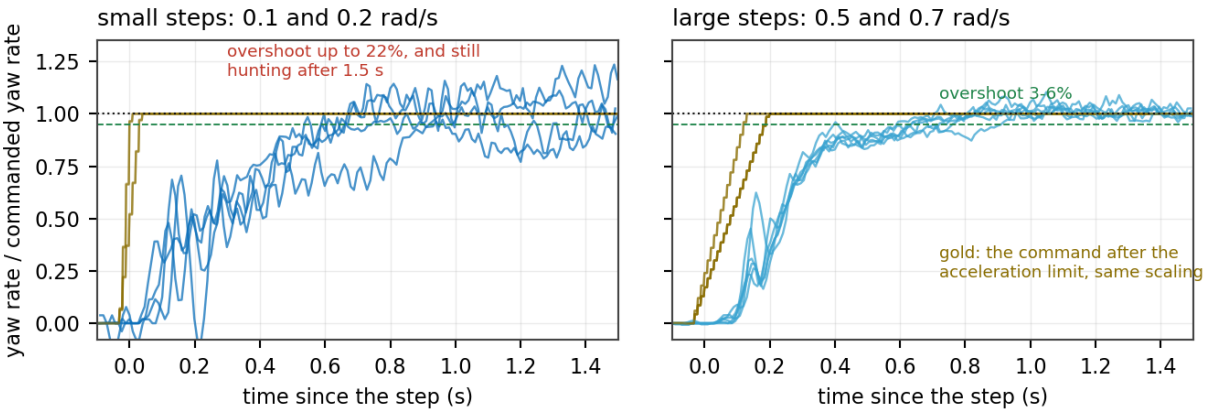


Figure 2. All ten rise edges, each divided by its own commanded value so they can be compared directly. Gold is the rate-limited command on the same scaling. The small steps are still hunting around the setpoint after 1.5 s; the large ones are settled by about 0.8 s.

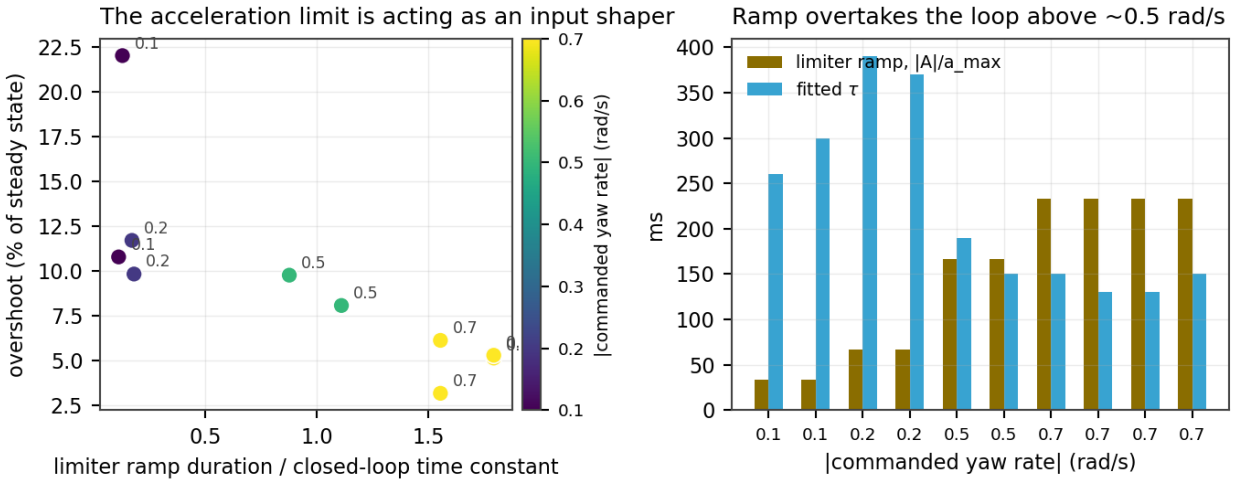


Figure 3. Left: overshoot against the ratio of limiter ramp duration to closed-loop time constant, coloured by commanded amplitude. Right: the two timescales side by side — the ramp only overtakes the loop above about 0.5 rad/s, which is exactly where the overshoot collapses.

Which signal is freshest, and why it is no longer the IMU

Cross-correlating each velocity channel against the raw wheel velocities over the whole 51 s run gives a lag per channel. Every one of these correlations is above 0.998, so the lags are well determined.

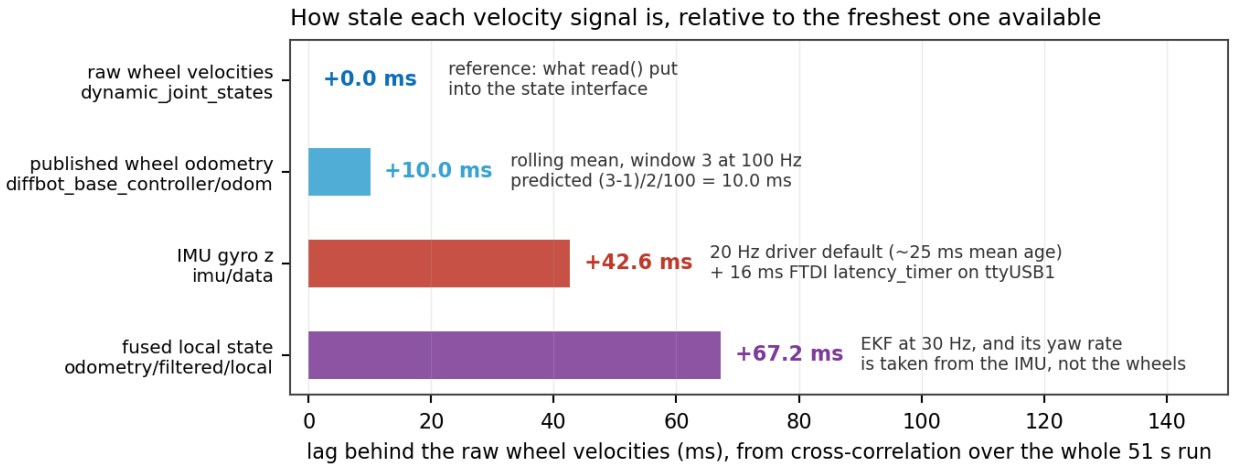


Figure 4. Lag behind the raw wheel velocities. The published odometry figure of +10.0 ms matches the predicted rolling-mean group delay of (3 – 1) / 2 / 100 = 10.0 ms to the first decimal place, which is a useful confirmation that nothing unexpected sits between the state interface and the odometry topic.

The ordering has reversed since latency_claude_1

In the course runs the raw wheels and the IMU were simultaneous to within 1 ms, and the *published* odometry sat 154 ms behind both because the rolling window was 10 at 30 Hz. The wheels were the slow signal and the gyro was the reference.

Now the published odometry is 10 ms behind the raw wheels, and the IMU is **42.6 ms behind the published odometry**. Wheel odometry has become the fastest state estimate on the robot, and the IMU has become the slow one. Nothing about the IMU changed; the wheels got roughly 200 ms faster and overtook it.

The IMU's 43 ms is not inherent to the sensor. Two configuration defaults account for nearly all of it:

- `um7_driver` declares `update_rate` with a default of 20, and `imprimis_hardware.launch.py` passes only `port`. The measured 19.6 Hz confirms the default is in force. A 20 Hz publication carries a mean sample age of about 25 ms before anything else happens to it.
- `/dev/ttyUSB1` is an FTDI bridge and its `latency_timer` reads 16 ms, the kernel default. That is 16 ms added to every packet that does not happen to fill a USB frame. (`/dev/ttyUSB0`, board A, is a CP2102 and exposes no such file, so this does not affect the drivetrain path.)

25 ms plus 16 ms is 41 ms against a measured 43 ms. Both are one-line changes and together they would recover most of it. The caveat worth stating is that the 20 Hz sampling also limits how precisely this lag can be resolved — a cross-correlation between a 100 Hz and a 20 Hz signal cannot distinguish internal filtering from publication rate — but the fix is the same under either explanation, and the EKF sees it as real lag regardless.

The EKF is now the slowest thing in the chain

`odometry/filtered/local` is **67 ms behind the published odometry and 77 ms behind the raw wheels**. That is the topic nav2 reads for the robot's velocity. The decomposition is straightforward: `Local_EKF.yaml` sets `frequency: 30.0`, and its yaw rate comes from `imu0` (index 11 true) rather than `odom0` (index 11 false). So it inherits the IMU's 43 ms and adds about 25 ms of its own sampling, which is what the measurement of 26 ms between the gyro and the EKF output shows.

The tempting conclusion is to take yaw rate from the wheels instead. That is probably the wrong move: the wheels still carry a scale error (next section) and they lie outright under slip, which is exactly the situation the gyro exists to cover. Fixing the IMU's rate and its latency timer, and raising the EKF frequency, gets the same latency back without giving up the gyro.

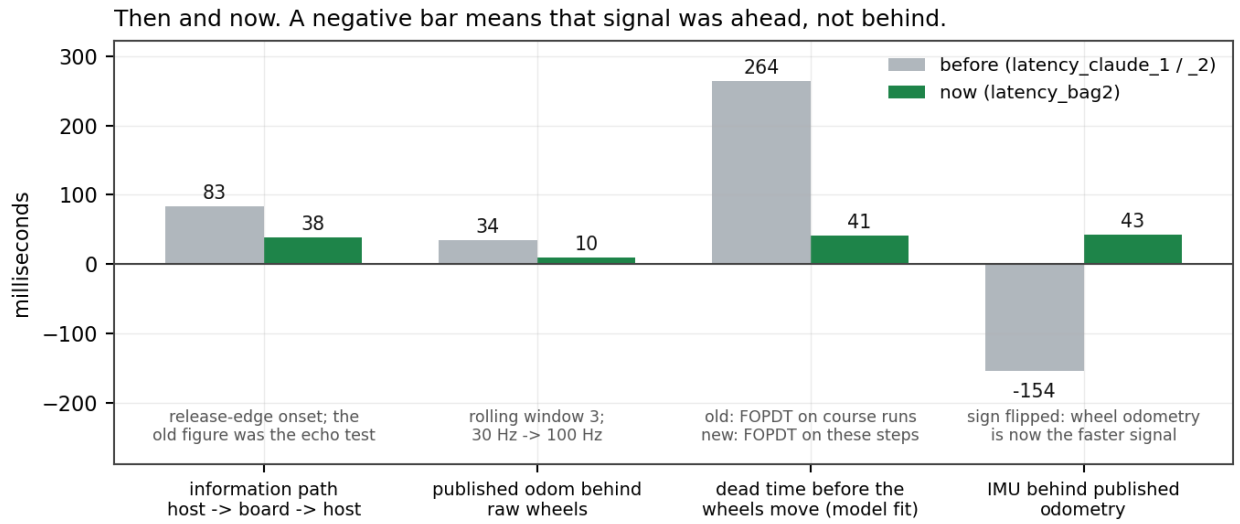


Figure 5. Then and now. The –154 ms bar is negative because at that time the IMU was *ahead* of the published odometry rather than behind it. The dead-time pair is the least directly comparable of the four: both are FOPDT fits, but the earlier one was taken from course runs at 30 Hz with a rolling window of 10, not from steps.

A kinematic check that came free with the test

Because every step was a pure in-place rotation, the gyro and the wheels are measuring the same quantity by two independent routes, and their ratio is a calibration.

	latency_claude_1 (course runs)	This test (|A| ≥ 0.2)
gyro / wheel-derived yaw rate	1.238 – 1.262	**1.0514 ± 0.0028** (n = 8)
`wheel_separation` in force	1.016 m	0.856 m, tape-measured
implied effective value	—	0.814 m

Correcting `wheel_separation` took a 24% disagreement down to 5%, and the residual is now consistent to ±0.3% across eight steps spanning 0.2 to 0.7 rad/s, which is a far tighter number than anything available before. The two 0.1 rad/s steps are excluded because at that amplitude the quantisation dominates; they read 1.032 and 0.987.

Do not change wheel_separation on the strength of this

Every one of these steps is an in-place spin, which is the one manoeuvre where the tyres scrub hardest and the effective contact points pull inward. A spin test will systematically report a narrower track than straight-line or gentle-arc driving does, so 0.814 m is a rotation-specific effective value and not a correction to a tape measure.

The right way to settle it is a figure-of-eight or a long constant-radius arc with the same gyro-versus-wheels comparison. If that also reports about 1.05, the parameter is worth revisiting; if it reports about 1.00, then 0.856 m is correct and the 5% seen here is scrub.

What this means for nav2

This is the part that connects back to the oscillation documented in the iteration 3 change report. That report concluded the oscillation on straights was not a software timing problem — all loops ran on time, plan churn was under 4° — and attributed it to actuation dead time and a kinematic scale error. Both of those have now been measured directly rather than inferred, and one of them has largely been fixed.

MPPI runs at `controller_frequency: 10.0` with `model_dt: 0.1`, so it issues a fresh command every 100 ms and its rollout model assumes the commanded yaw rate is reached within one model step. Here is what actually happens in that time, averaged over the ten steps:

Time since the command changed	100 ms	200 ms	300 ms	500 ms	700 ms	1000 ms
Fraction of the commanded yaw rate achieved	13%	34%	66%	84%	94%	100%

After one full MPPI model step the robot has realised about an eighth of the yaw-rate change it was asked for. MPPI has no dead-time term and no actuator model, so it sees an under-response, commands more, and then has to unwind the surplus once the plant catches up. That is a textbook mechanism for the hunting behaviour seen on straights, and the 41 ms dead time is only a small part of it — the 130–150 ms closed-loop time constant and the 233 ms limiter ramp matter more at MPPI's timescale.

Three specific mismatches follow, none of which are changed in this report:

- `az_max: 2.0` in MPPI against `angular.z.max_acceleration: 3.0` in diff_drive_controller. This mismatch has reversed sign since iteration 1, when MPPI was the optimistic one at 2.0 against 1.5. MPPI now plans for a slower turn-in than the base will actually deliver, so its rollouts understate its own agility. Aligning `az_max` to 3.0 is the obvious change, but it should be made alongside the overshoot test described above, because the ramp is currently suppressing overshoot and loosening it has a cost.
- `model_dt: 0.1` against a plant whose dead time plus time constant is roughly 190 ms for large yaw steps. The rollout cannot represent that with a single step of lag even in principle.
- The low-amplitude response is the weak spot, and it is the regime nav2 lives in. On a straight, MPPI commands small yaw corrections continuously — 0.1 to 0.2 rad/s, exactly the two amplitudes here that take 800–900 ms to converge and overshoot by 10–22%. The large-step performance is good and is not what the robot spends its time doing.

Found outside Course2027.yaml and recovery_nav_bt.xml

Per the standing instruction, these are recorded and not changed. They are ordered by how much they are worth.

#	Where	What	Why it matters
1	`imprimis_hardware.launch.py`, the `um7_driver` node	Only `port` is passed, so `update_rate` takes the driver's default of 20 Hz.	About 25 ms of the IMU's 43 ms lag. The UM7 supports far higher rates; 100 Hz fits inside the driver's default 115200 baud with margin.
2	`/dev/ttyUSB1` (udev)	FTDI `latency_timer` is 16 ms, the kernel default.	The other ~16 ms. A udev rule setting it to 1 is the standard fix and costs nothing.
3	`Local_EKF.yaml`	`frequency: 30.0`, and yaw rate is taken from `imu0` rather than `odom0`.	Adds ~25 ms on top of the IMU's own lag, and the EKF output is what nav2 reads. Raising the frequency is the safe half of this; moving yaw rate to the wheels is not recommended while they still carry a scale error.
4	`diffbot_system.cpp`, `read()`	On a failed or empty read the previous velocity is silently reused, and there is no counter.	15.8% of wheel reports while moving are byte-identical to the previous one, in runs of up to 5 cycles. Most of that is almost certainly a genuinely unchanged encoder count at constant speed, but there is no way to tell from the outside. A dropped-read counter on the existing gpio state interface would settle it permanently.
5	`diffbot_system.cpp`, `write()`	The failure branch is still an empty block with a comment in it.	A short or failed write leaves board A with a truncated line. Unchanged since latency_claude_2 and still worth a log throttle at minimum.
6	`config.h` / gpio state interface	`rtlatency` is dead, as *latency_real_report_2* says.	It still occupies a state interface and is still bagged. Repurposing it as the dropped-read counter from #4 would cost nothing and require no new plumbing.
7	Working tree, `boardB.cpp`	B's PID loop, the encoder PCNT setup, the timeout stop and the serial init are all currently commented out, and B returns the received setpoint instead of `getAngvel()`.	This is the scope-test configuration and is *not* what produced latency_bag2 — the bagged data is unmistakably real encoder data, quantised and noisy. Worth making sure it is not flashed or committed by accident.
8	`/rosout` during the run	61 `Velodyne poll() timeout` messages over 51 s.	The LiDAR produced nothing for the whole bag. Harmless for a bench test, but it means this bag cannot be reused for anything perception-related.

One earlier item can be closed: the serial read timeout was zero in *latency_claude_2* and is now 0.003 s in `on_activate`. The worst observed cycle in this bag was 11.47 ms, so it is not costing anything in practice. The one residual subtlety is that `readline` applies the timeout per `read_()` call and reads one byte per call, so the budget is nominally per byte rather than per line; it only bites if a line arrives fragmented, which did not happen here.

Method and caveats

- Everything is measured from the bag receive timestamp of `cmd_vel`, because rqt leaves the header stamp unset. For the other channels the header stamp and the receive time agree to within 0.25 ms, so the choice does not matter for them.
- Onset detection requires the signal to cross the threshold and stay across for 2 of the next 3 samples, which rejects single-sample blips. The threshold sweep under *Informational latency* is the honest statement of how much that choice is worth: about 35 ms across the plausible range.
- The FOPDT fit is a grid search over dead time (0–300 ms in 2 ms steps) and time constant (20–1200 ms in 10 ms steps), with the plant input taken as the command after a simulated rate limiter running on the

controller's own 100 Hz tick. Fitting against the raw step instead would fold the ramp into the dead time and report something two to five times too large — that is the same trap documented in *latency_claude_2*, and it is why the naive cross-correlation figure from that report was 304 ms against a true 112 ms.

- n = 10 for every rise statistic and n = 10 for every release statistic. The spreads quoted are sample standard deviations, not standard errors; with ten samples the mean is itself uncertain by roughly a third of the quoted sd.
- Tools used, all in `claude_tools/`: `step_latency3.py` (edges, onsets, threshold sweep, cross-correlation lags), `make_figures_latency3.py`, `make_report_latency3.py`. `step_latency3.py` takes `--a-max`, `--rate`, `--window` and `--onset-quanta`, so the next step test can be compared against this one directly by re-running it with the new numbers.